

```

    "assistant": "Ajit"
  },
  {
    "id": "A3",
    "author": "Sir Arthur Conan
Doyle",
    "assistant": "Watson"
  },
  {
    "id": "A4",
    "author": "Agatha Christie",
    "assistant": "Hastings"
  }
]

```

6. Create a DAO layer in the project folder as *db.js*:

```

const { DataStore } =
require('notarealdb');

const store = new DataStore('./data');

module.exports = {
  detectives:store.
collection('detectives'),
  authors:store.collection('authors')
};

```

7. Create a schema file as *schema.graphql* in the project folder:

```

type Query {
  test: String,

  detectives:[Detective],
  detectiveById(id:ID!):Detective,

  authors: [Author],
  assistantByDetective(id:ID!):Auth
or,
}

type Detective {
  id:ID!
  name:String
  address:String
  authorId:String
}

type Author {
  id: ID!

```

```

author:String
assistant:String
}

```

8. Create the resolver as *resolvers.js*:

```

const db = require('./db')
const Query = {
  test: () => 'Congrats!, GraphQL
server is up & running !!',

  detectives: () => db.detectives.
list(),
  detectiveById: (root, args,
context, info) => {
    return db.detectives.get(args.
id);
  },

  authors: () => db.authors.list(),
  assistantByDetective: (root, args,
context, info) => {
    return db.authors.get((db.
detectives.get(args.id)).authorId);
  },
}
module.exports = { Query }

```

The resolver is very important. This is where we define the different methods that will be called directly by the clients to route the requests to multiple tables to get data; e.g., there are *GetAll* methods like *detectives* and *authors*. Please look at the *assistantByDetective* method. We purposefully kept the 'assistant' (e.g., Dr Watson for Sherlock Homes) in the *Author* schema just for demonstration purposes. The *assistantByDetective* is taking the *detective's Id* and getting detective schema *db.detectives.get(args.id)*. The detective schema has the *authorId*. This *authorId* is used to get the detailed author schema looking into the *db.authors* list by *db.authors.get(...)*.

9. Finally, create the *server.js* to host the node app:

```

const bodyParser = require('body-

```

```

parser');
const cors = require('cors');
const express = require('express');
const db = require('./db');

const port = process.env.PORT || 8000;
const app = express();

const fs = require('fs')
const typeDefs = fs.readFileSync('./
schema.graphql',{encoding:'utf-8'})
const resolvers = require('./
resolvers')

const {makeExecutableSchema} =
require('graphql-tools')
const schema =
makeExecutableSchema({typeDefs,
resolvers})

app.use(cors(), bodyParser.json());

const {graphqlExpress,graphqlExpress}
= require('apollo-server-express')
app.use('/graphql',graphqlExpress({sch
ema}))
app.use('/graphql',graphqlExpress({en
dpointURL:'/graphql'}))

app.listen(
  port, () => console.info(
    `Server started on port ${port}`
  )
);

10. The my_app dir will look something
like this:

λ dir

Directory of my_app

10/02/2020 11:23 AM <DIR> .
10/02/2020 11:23 AM <DIR> ..
10/02/2020 10:51 AM <DIR> data
10/02/2020 10:58 AM 199 db.js
10/02/2020 10:46 AM <DIR> node_modules
10/02/2020 10:46 AM 102,417 package-
lock.json
10/02/2020 10:46 AM 415 package.json
10/02/2020 11:22 AM 112 resolvers.js
10/02/2020 11:17 AM 35 schema.

```